How to Order your Tasks: Investigating Task Sequencing in Continual Learning

Riccardo Rigoni Guiv Farmanfarmaian Ashwin Shenai

Abstract

Catastrophic forgetting, where models lose prior knowledge while learning new tasks, remains a major challenge in continual learning. This study investigates the role of task grouping and ordering on continual learning performance. We enhance a task similarity metric, and investigate various task grouping and ordering strategies to minimize forgetting. Experiments on MNIST-10 reveal that grouping similar tasks together and minimizing inter-group similarity reduces forgetting by stabilizing feature representations. Results on CIFAR-50[1] are more ambiguous.

1. Introduction

A central challenge in continual learning is catastrophic forgetting, where a model's performance on earlier tasks deteriorates as it learns new ones. This work investigates the hypothesis that the performance of a continual learner is influenced by which tasks are grouped and learned and the order in which they are presented.

Specifically, we assume a set of n tasks can be learned in batches of size k. A model $\mathcal{M}_\theta$ must learn a set of tasks $\mathcal{T} = \{T_1, .., T_n\}$. The model learns k tasks at a time. More precisely, we consider T_i being an object/digit classification task. We first break the task set into pairwise disjoint task groups $\tau_1 \cup \tau_2 \cup \cdots \cup \tau_m = \mathcal{T}$, where each task group or cluster τ_j is of size k. For each group τ_j, the model minimizes a loss function $\ell_j(\theta_j)$ on the combined training data of the tasks in that task group. The continual learning objective requires the model to also simultaneously minimize previous task group losses $l_1(\theta_1), l_2(\theta_2), \ldots, l_{j-1}(\theta_{j-1})$. However, we ignore this additional objectives and consider a naive learner. Under this setting, we investigate the two aspects of task ordering: i) Grouping: Constructing the k-subsets based on a task-level metric, and ii) Ordering: Constructing the learning schedule by ordering the task groups.

[1]We only use the first 50 classes from the CIFAR-100 dataset due to resource constraints.

The code is made available at: https://github.com/ashwin2802/TOCL_DL2024

Task ordering in continual learning can consider two primary paradigms: curriculum learning, which sequences tasks based on their difficulty, and task similarity, where tasks with shared characteristics are grouped together. The former doesn't provide substantial improvements in traditional settings with proper parameterization (Wu et al. (2021); Mannelli et al. (2024)). On the other hand, the latter can significantly affect performance, as tasks with intermediate similarity lead to increased forgetting (Ramasesh et al., 2020). Furthermore, Bell & Lawrence (2022) demonstrate that task sequence ordering significantly affects continual learning outcomes. They employ a similarity metric based on the curvature of the loss surface between task minima and gradient steps to calculate the similarity of tasks. We build on their work exploring both alternative similarity metrics, as well as task grouping.

Indeed, Mirzadeh et al. (2020) demonstrated that the solution of a multi-task learning problem is often linearly connected to the solution of a continual learning problem for the same tasks. This motivated our exploration of grouping tasks into clusters a given size k. Multi-task learning (MTL) trains models on multiple tasks simultaneously, by jointly optimizing shared representations. This helps improve generalization across tasks (Caruana, 1997). Drawing inspiration from this learning paradigm, we propose grouping tasks into disjoint subsets and investigating how the ordering of these groups affects catastrophic forgetting.

Finally, we explore the effect that our methods have on the change in the hidden representations of previously learned tasks.

2. Methodology

In this section, we describe the task similiarity metrics which we use to construct the task groups, followed by the task group ordering strategies and the metrics used to evaluate the performance of the model across all the tasks.

2.1. Measuring Task Similiarity

To cluster and order tasks based on their similarity, we employ clustering methods based on the following similarity metrics. Each model is trained independently on a task T_i from the same initialization. For each task T_i, the model

parameters Θ_i are optimized to minimize the task-specific loss $l_i(\Theta_i)$.

c-Metric - Curvature-based Metric: The curvature-based metric, introduced in Ramasesh et al. (2020), quantifies task interference by measuring the curvature of the loss surface at a task's minima in the direction of a new task. Higher curvature indicates greater interference and task dissimilarity. See Appendix A.1 for details.

d-Metric - Gradient-Alignment Distance: Building on the curvature-based metric, we incorporate gradient alignment to capture additional information about task similarity. For tasks T_i and T_j, let $\mathbf{g}_i = \nabla l_i(\Theta)$ and $\mathbf{g}_j = \nabla l_j(\Theta)$ represent the gradients of the loss functions with respect to the model parameters. The gradient alignment is defined as:

$$g_{i,j} = \mathbf{g}_i^T \mathbf{g}_j.$$

Negative values of $g_{i,j}$ indicate task interference (likely to increase loss), while positive values reflect aligned updates that reduce loss. Preliminary results suggested that $g_{i,j}$ alone is insufficient to capture the similarity of the task. Thus, we combine it with the curvature metric $c_{i,j}$ to define am enhanced similarity metric $d_{i,j}$:

$$d_{i,j} = c_{i,j} - \left[\text{mag}_{10}(c_{i,j}) + 1\right] \cdot g_{i,j}.$$

where $\text{mag}_{10}(x) = \lfloor \log_{10} |x| \rfloor$ scales the magnitude of $c_{i,j}$ to balance the contributions of the curvature and gradient components. The negative sign on $g_{i,j}$ ensures that aligned gradients reduce $d_{i,j}$, indicating greater task similarity.

2.2. Clustering Methods

Tasks were grouped into subsets of size k using heuristic methods like Hierarchical Clustering, as exact approaches such as Linear Programming were computationally infeasible. The following clustering strategies were implemented and compared:

- **$\mathbf{C_{max}}$ - Maximum Similarity Clusters:** Tasks within each cluster were selected to maximize intra-cluster similarity, grouping tasks that are most similar.
- **$\mathbf{C_{min}}$ - Minimum Similarity Clusters:** Clusters were constructed by minimizing intra-cluster similarity, encouraging diversity among tasks within each group.
- **B - Random Clusters Baseline:** Tasks were assigned to clusters randomly, serving as a baseline for evaluating the impact of task similarity on forgetting and accuracy metrics.

2.3. Sequencing Strategies

Given a set of n tasks T_i, grouped into $\lceil n/k \rceil$ clusters $\{\tau_1, \tau_2, \ldots, \tau_{n/k}\}$, where each cluster τ_i contains k tasks, we aim to determine the optimal ordering of these clusters to enhance continual learning performance. Let $\mathbf{M} \in \mathbb{R}^{n \times n}$ represent the task similarity matrix, where $M_{i,j}$ denotes the (dis-)similarity between tasks T_i and T_j. Note that by definition of our metrics, higher $M_{i,j}$ values denote lower similarity. The strategies considered are described as follows:

G1 - Min Cut: aims to minimize the total similarity across adjacent clusters in a given permutation. For a permutation π of the clusters, the cost is defined as:

$$\text{Cost}(\pi) = \sum_{i=1}^{n/k-1} \sum_{T_a \in \tau_{\pi(i)}} \sum_{T_b \in \tau_{\pi(i+1)}} M_{a,b},$$

where $C_{\pi(i)}$ denotes the i-th cluster in the permutation π. The optimal ordering π^* is determined by maximizing the cost:

$$\pi^* = \arg\max_{\pi} \text{Cost}(\pi).$$

G2 - Max Cut: This strategy is analogous to **G1 - Min Cut** but instead seeks to maximize the total similarity across adjacent clusters.

G3 - Min Cumulative Cut: This strategy considers all pairs of clusters, not just adjacent ones, aiming to minimize the cumulative similarity across the entire permutation. The cost for a permutation π is defined as:

$$\text{Cost}(\pi) = \sum_{i=1}^{n/k} \sum_{j=i+1}^{n/k} \sum_{T_a \in \tau_{\pi(i)}} \sum_{T_b \in \tau_{\pi(j)}} M_{a,b}.$$

The optimal ordering π^* is determined by maximizing this cumulative cost:

$$\pi^* = \arg\max_{\pi} \text{Cost}(\pi).$$

G4 - Max Cumulative Cut: This strategy is analogous to **G3 - Min Cumulative Cut** but seeks to maximize the cumulative similarity across all pairs of clusters.

2.4. Models and Datasets

We used a 3-layer MLP (256 hidden units, ReLU activation) and ResNet-18 (He et al., 2015), adapted to Multi-Task Learning (MTL) with a shared feature extractor and task-specific output heads, resembling a task-incremental, model-incremental setup.

Naive Strategy: Models were trained with vanilla SGD, without meta continual learning algorithms (e.g., iCaRL (Rebuffi et al., 2017)), as a baseline for task grouping and ordering. For similarity matrix computation, the same initialization was trained for 10 epochs on a task of distinguishing class/digit from noise.

Across experiments, the MLP and ResNet-18 are trained for 20 and 30 epochs, respectively, with cross entropy loss and Adam optimizer (learning rate: 0.0001, betas: (0.9, 0.999), weight decay: 10^{-4}).

2.5. Performance Metrics

We evaluate the model's performance using several standard metrics commonly employed in continual learning. The main metrics are the Average Accuracy and the Average Forgetting Measure. Detailed definitions of these metrics are in the Appendix A.2.

2.6. Hidden Representation Analytics

Let π be an ordering of the clusters. We assume for simplicity that π is the identity permutation and the order is $\{\tau_1, \ldots, \tau_l\}$. We denote the average hidden representation, of the final layer before the classifiaction head, on the validation of set τ_i after learning cluster i as $z_{i,i} \in \mathbb{R}^d$. We desire to analyze how much this average embedding changes between subsequent timesteps $z_{i,j}$, and $z_{i,j+1}$.

One measure is the Euclidean Distance between these consecutive embeddings. We define the **Trajectory Length** L_i as the average Euclidean distance between consecutive embeddings of cluster τ_i over all timesteps, i.e.

$$L_i = \sum_{j=1}^{l-1} \|z_{i,j+1} - z_{i,j}\|_2.$$

Another metric is the variance, which measures how the embeddings $z_{i,j}$ deviate from their mean μ_i. The mean embedding μ_i is computed from timesteps 1 to timesteps l. The **variance** is then defined as:

$$\text{Var}_i = \frac{1}{l-1} \sum_{k=1}^{l} \|\mathbf{z}_{i,k} - \mu_i\|_2^2.$$

3. Experiments and Results

We explore the impact of task grouping and ordering on letter recognition (MNIST-10 w/ MLP) and object classification (CIFAR-50 w/ ResNet-18). [2]

3.1. Effect of Task Grouping

This experiment evaluates the impact of different task grouping strategies and similarity metrics on forgetting and accuracy. We average the results over 10 runs with random task ordering, to isolate the impact of grouping. The results are presented in Table 1 for MNIST-10 with MLP and Table 2 for CIFAR-50 with ResNet-18.

[2]We only use the first 50 classes from the CIFAR-100 dataset due to resource constraints.

Metric	B	C_{max}, c	C_{max}, d	C_{min}, c	C_{min}, d
A_5(% ↑)	79.8	82.7	78.9	82.8	**85.4**
F_5(% ↓)	24.3	20.3	25.1	20.8	**17.2**

Table 1. Average final accuracy and forgetting on MNIST-10 with MLP and 2-clusters. B: baseline; C_{max}, C_{min}: clustering strategy; c, d: similarity metrics, as in (§2).

Metric	B	C_{max}, c	C_{max}, d	C_{min}, c	C_{min}, d
A_6(% ↑)	40.7	**42.1**	40.1	38.6	41.2
F_6(% ↓)	49.6	**49.3**	49.8	50.4	49.4

Table 2. Average final accuracy and forgetting on CIFAR-50 with ResNet-18 and 8-clusters. B: baseline; C_{max}, C_{min}: clustering strategy; c, d: similarity metrics, as in (§2).

The results suggest that clustering strategies such as (C_{max}, c) and (C_{min}, d) outperform random grouping across architectures and datasets.

3.2. Effect of Task Ordering

In this experiment, we aim to further mitigate forgetting by employing optimal task ordering strategies. We evaluated various ordering strategies applied to optimal groupings from the previous part.

For MNIST-10, Table 3 highlights that maximizing similarity across adjacent tasks significantly reduces forgetting and improves accuracy, particularly in combination with the (C_{max}, d) grouping. Notably, (C_{max}, d) performed poorly with random ordering (see Table 1), indicating that the combination of appropriate task grouping and optimal ordering is critical for mitigating forgetting.

For CIFAR-50 with ResNet-18, Table 4 shows that different ordering strategies perform best depending on the grouping and metric. For instance, (C_{min}, d) does not benefit from any of the considered ordering strategies, performing on par with random ordering. In contrast, for (C_{max}, c), the G3 ordering, which minimizes cumulative similarity between clusters, achieves the best reduction in forgetting. We conclude that there is no consistent pattern between the specific ordering that works best with a given grouping strategy. Also, there seems to be no optimal grouping strategy across datasets and architectures.

3.3. Intermediate Representation Stability

We evaluated the stability of intermediate representations during training using the metrics in Section 2.5, examining their relationship with task ordering and forgetting mitigation.

For the MLP on MNIST, Table 5 shows a strong correlation

(Grouping, Metric)	Ordering	$A_5(\% \uparrow)$	$F_5(\% \downarrow)$
(C_{max}, c)	-	82.7	20.3
(C_{max}, d)	**G2**	**91.6**	**9.4**
(C_{min}, c)	G3	86.8	15.7
(C_{min}, d)	G2	87.6	14.3

Table 3. Comparison of best task ordering in terms of forgetting F_5 across grouping and metric pairs on MNIST-10 with MLP and 2-clusters.

(Grouping, Metric)	Ordering	$A_6(\% \uparrow)$	$F_6(\% \downarrow)$
(C_{max}, c)	**G3**	**44.3**	**46.5**
(C_{max}, d)	G2	41.27	49.6
(C_{min}, c)	G2	39.74	49.52
(C_{min}, d)	G4	40.6	49.3

Table 4. Comparison of best task ordering in terms of forgetting F_6 across grouping and metric pairs on CIFAR-50 with ResNet-18 and 8-clusters.

between reduced Trajectory Length and Variance and lower forgetting, highlighting how task grouping and ordering stabilize intermediate representations.

For ResNet-18, while most observations followed this trend, the best-performing configuration in terms of forgetting showed higher Trajectory Length and Variance, deviating from the expected pattern. Additional details, including tables and distribution plots, are provided in Appendix B.2.

These findings suggest that stabilizing intermediate representations mitigates forgetting in simpler architectures like MLPs, but deeper models like ResNet-18 may rely on more complex or nuanced mechanisms, warranting further investigation.

4. Discussion

This study explored the impact of task grouping and ordering on mitigating catastrophic forgetting in continual learning. Experiments on MNIST-10 (MLP) and CIFAR-50 (ResNet-18) showed that while grouping and ordering improve performance over the baseline, their effectiveness varies by architecture.

4.1. Task Grouping and Ordering

Grouping tasks with similarity-based metrics reduced forgetting for both architectures. For MNIST-10, (C_{min}, d) performed best, while for CIFAR-50, improvements were less consistent. Optimal ordering further reduced forgetting, particularly for MNIST-10, where G2 (Max Cut) with (C_{max}, d) achieved the best results. For CIFAR-50, G3 (Min Cumulative Cut) with (C_{max}, c) showed modest improvements.

(Group., Metric, Ord.)	$F_6(\% \downarrow)$	Trajec	Var
(B)	24.3	32.79	6.97
(C_{max}, c)	20.3	31.55	6.47
Δ (Baseline vs C_{max} Clust.)	**-4.0**	**-1.24**	**-0.50**
(C_{max}, d, -)	25.1	34.50	7.16
(C_{max}, d, G_{opt})	9.4	36.44	7.09
Δ (Rand. vs. Opt. Order)	**-15.7**	1.94	**-0.07**
(C_{min}, c, -)	20.7	30.59	6.23
(C_{min}, c, G_{opt})	15.7	29.29	5.76
Δ (Rand. vs. Opt. Order)	**-5.0**	**-1.30**	**-0.47**
(C_{min}, c, -)	20.7	30.59	6.23
(C_{min}, c, G4)	25.0	31.55	6.37
Δ (Rand. vs. Opt. Order)	4.3	0.96	0.14
(C_{min}, d, -)	17.3	31.08	6.39
(C_{min}, d, G_{opt})	14.7	29.17	6.00
Δ (Rand. vs. Opt. Order)	**-2.6**	**-1.91**	**-0.39**

Table 5. Comparison of task ordering in terms of forgetting (F_6), average trajectory length and average variation, across grouping and metric pairs on MNIST-10 with MLP and 2-clusters. G_{opt} represents the optimal task ordering for the corresponding (Grouping, Metric) pair that achieves the lowest forgetting. The Δ rows explicitly show changes relative to the more general case.

4.2. Representation Stability

For MLP, reduced Trajectory Length and Variance correlated strongly with lower forgetting, demonstrating the importance of stabilizing intermediate representations. In contrast, no consistent correlation was observed for ResNet-18, indicating that deeper architectures may mitigate forgetting through mechanisms beyond representation stability.

4.3. Limitations and Future Work

While grouping and ordering strategies reduced forgetting, further investigation is needed to ensure their impact on ResNet-18. Future work should explore additional architectures and scalable methods to enhance the generalizability of these findings. Novel architectures such as Kolmogorov-Arnold Networks (KANs) could offer valuable insights into the interplay between representation stability and forgetting. In this study, we prioritized understanding and explaining the performance improvements achieved with the MLP and ResNet-18 architectures, leaving the exploration of KANs for future research. A final drawback is that the initial tasks T_i are learned by classifying the corresponding digits or images versus random noise. This might explain why other explored similarity metrics such as Centered Kernel Alignment and the similarities of the diagonals of the Fisher Information Matrix failed to yield improvements.

References

Bell, S. J. and Lawrence, N. D. The effect of task ordering in continual learning, 2022. URL https://arxiv.org/abs/2205.13323.

Caruana, R. Multitask learning. *Machine Learning*, 28(1): 41–75, 1997. doi: 10.1023/A:1007379606734.

De Lange, M., Aljundi, R., Masana, M., Parisot, S., Jia, X., Leonardis, A., Slabaugh, G., and Tuytelaars, T. Embracing change: Continual learning in deep neural networks. *Trends in Cognitive Sciences*, 24(12):1028–1040, 2020.

He, K., Zhang, X., Ren, S., and Sun, J. Deep residual learning for image recognition, 2015. URL https://arxiv.org/abs/1512.03385.

Mannelli, S. S., Ivashynka, Y., Saxe, A., and Saglietti, L. Tilting the odds at the lottery: the interplay of overparameterisation and curricula in neural networks, 2024. URL https://arxiv.org/abs/2406.01589.

Mirzadeh, S., Farajtabar, M., Görür, D., Pascanu, R., and Ghasemzadeh, H. Linear mode connectivity in multitask and continual learning. *CoRR*, abs/2010.04495, 2020. URL https://arxiv.org/abs/2010.04495.

Ramasesh, V. V., Dyer, E., and Raghu, M. Anatomy of catastrophic forgetting: Hidden representations and task semantics, 2020. URL https://arxiv.org/abs/2007.07400.

Rebuffi, S.-A., Kolesnikov, A., Sperl, G., and Lampert, C. H. icarl: Incremental classifier and representation learning, 2017. URL https://arxiv.org/abs/1611.07725.

Wu, X., Dyer, E., and Neyshabur, B. When do curricula work?, 2021. URL https://arxiv.org/abs/2012.03107.

A. Methodology

A.1. Curvature-based Loss

The curvature-based metric, introduced in Bell & Lawrence (2022) measures the interference between tasks by measuring the curvature of the loss landscape. It evaluates the change in the loss of one task when parameters are adapted to another task. Higher curvature values indicate greater interference and lower task similarity and minimal interference.

The curvature-based metric measures the change in the loss of current tasks caused by introducing a new task. Higher curvature values indicate greater interference and dissimilarity, while lower values suggest minimal loss changes and higher task similarity. how that this metric works in practice.

Following their approach, we train the model individually on each task T_i from the same initialization. That is, for each task T_i, we have the model parameters Θ_i where the loss $l_i(\Theta_i)$ is minimized. Then, for each task T_j where $i \neq j$, we compute the curvature in the direction of the new task:

$$\begin{aligned} c_{ij} &= \mathbf{g}_j^T \mathbf{H} \mathbf{g}_j \\ \mathbf{H} &= \nabla^2_{\Theta_i} l_i(\Theta_i) \\ \mathbf{g}_j &= \nabla_{\Theta_i} l_j(\Theta_i), \end{aligned}$$

where the gradient $\mathbf{g}_j$ is computed on a batch of samples from the task T_j.

A.2. Performance Metrics

In the following, let σ denote a task group ordering, and θ_j be the model parameters after the jth task group $\tau_{\sigma(j)}$ is learned. In line with the literature on continual learning (De Lange et al., 2020), we consider the following metrics:

Average Accuracy: The average accuracy over all tasks learned until timestep j is defined as:

$$\mathrm{A}_j = \frac{1}{j} \sum_{i=0}^{j} \sum_{k|T_k \in \tau_i} a_k(\theta_i), \tag{1}$$

where a_k is the validation accuracy on Task T_k. This metric provides a measure of overall performance across all tasks learned so far.

Average Forgetting:

$$\mathrm{F}_j = \frac{1}{j-1} \sum_{i=1}^{j-1} \sum_{k|T_k \in \tau_i} \max_{l \in \{0,\ldots,j-1\}} a_k(\theta_l) - a_k(\theta_j), \tag{2}$$

which quantifies the average maximum loss in accuracy for any of the learned tasks compared to their best observed accuracy.

Average Incremental Accuracy:

$$\mathrm{IA}_j = \frac{1}{j} \sum_{i=0}^{j} \mathrm{A}_i, \tag{3}$$

which computes the average of cumulative accuracy across all tasks learned up to step j, reflecting how well the model retains performance incrementally.

Average Backward Transfer:

$$\mathrm{BT}_j = \frac{1}{j-1} \sum_{i=1}^{j-1} \sum_{k|T_k \in \tau_i} \left[a_k(\theta_j) - a_k(\theta_i) \right], \tag{4}$$

which measures the influence of learning new tasks on previously learned tasks. A positive value indicates improvement in performance on earlier tasks.

(Group., Metric, Ord.)	$F_6(\% \downarrow)$	Trajec	Var
(B)	49.6	36.38	4.80
(C_{max}, d)	49.9	35.84	4.71
Δ (Baseline vs C_{max}, d)	**0.3**	**-0.54**	**-0.09**
(B)	49.6	36.38	4.80
(C_{min}, c, -)	50.37	39.44	5.19
Δ (Baseline vs C_{min}, c)	**0.77**	**3.06**	**0.39**
(B)	49.6	36.38	4.80
(C_{min}, d, -)	49.44	43.92	4.95
Δ (Baseline vs C_{min}, d)	**-0.16**	**7.54**	**0.15**
(B)	49.6	36.38	4.80
(C_{max}, c, -)	49.3	34.99	4.61
Δ (Baseline vs C_{max}, c)	**-0.30**	**-1.39**	**-0.19**
(C_{max}, c, -)	49.3	34.99	4.61
(C_{max}, c, G_{opt})	46.5	35.26	4.68
Δ (Rand. vs G_{opt})	**-2.80**	**0.27**	**0.07**

Table 6. Comparison of task ordering in terms of forgetting (F_6), average trajectory length, and average variance across grouping and metric pairs on CIFAR-50 with ResNet-18 and 8-clusters. G_{opt} represents the optimal task ordering for the corresponding (Grouping, Metric) pair that achieves the lowest forgetting.

B. Experiments and Results

B.1. Effect of Task Grouping

Below is the full analysis of our experiments on the effect of task grouping.

For MNIST-10 using the MLP, clustering based on maximizing intra-cluster similarity (C_{max}) achieves the best results when using the curvature-based metric (c). Conversely, minimizing intra-cluster similarity (C_{min}) performs better with the gradient-augmented metric (d), with this combination yielding the lowest forgetting and highest accuracy.

On CIFAR-50 with ResNet-18, the performance trends differ. Specifically, (C_{max}, d) and (C_{min}, c) increase forgetting and reduce accuracy. The best configuration for ResNet-18 is (C_{max}, c), though it provides less improvement compared to the optimal configuration for MLP on MNIST-10.

B.2. Intermediate Representations Stability

The behavior of the stability of the hidden representations on ResNet-18 was highly unusual. Thus, we investigated them further. We plotted the forgetting measure values at the last timestep against the average trajectory lengths and the average variance amongst all clusters. We discovered that with the exception of certain outliers, tasks with a lower forgetting measure typically have more stable intermediate representations. However, the best performing combinations (i.e. those with the lowest forgetting measure) exhibit an unusual behavior where their intermediate representations are far more volatile than those of the others. This phenomenon ought to be further investigated in future work.

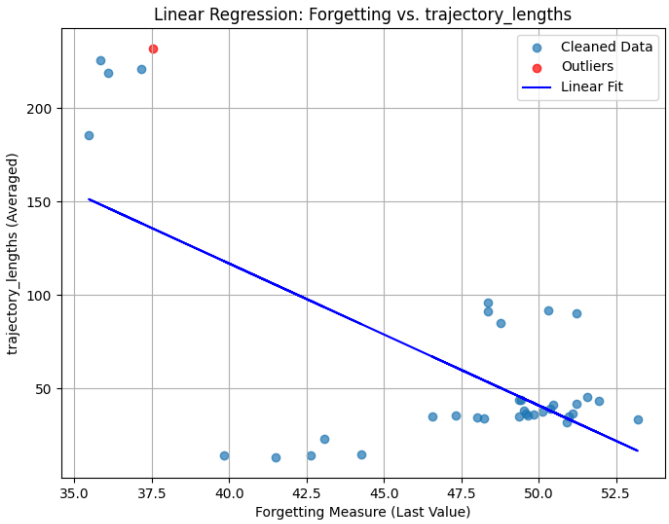

Figure 1. Forgetting Measure (Last Values) vs. Average Trajectory Lengths on ResNet-18.

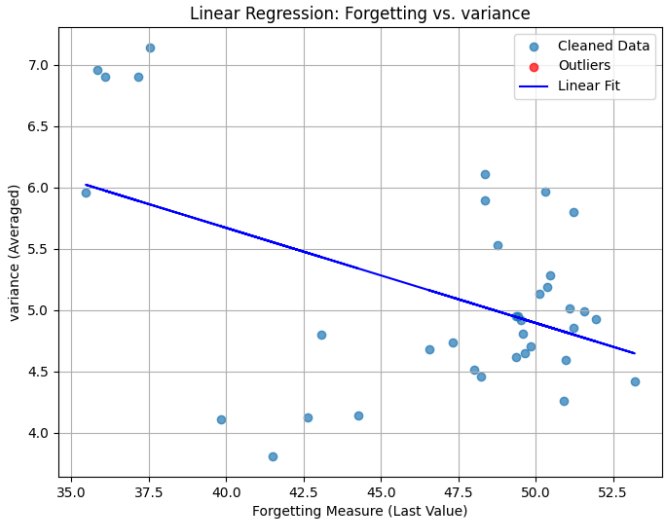

Figure 2. Forgetting Measure (Last Values) vs. Average Variance on ResNet-18